

A 4 Fős Csapat Részletes Szerepkörei és Munkamegosztása

EV Charging Platform & Management System — Fejlesztési Útmutató (1. Rész)

A projekt hatékony megvalósítása érdekében a 4 fős fejlesztőcsapat szigorú funkcionális és rétegalapú munkamegosztás szerint dolgozik. Minden fejlesztő egy-egy kritikus szoftverkomponensért felel, biztosítva a moduláris felépítést, a tiszta kódelvű (*clean code*) architektúrát és a párhuzamos, ütközésmentes GitHub workflow-t a dev ágon.

1. Fejlesztő: Architektúra, Alapkonfiguráció és Adatbázis

Java 17/21

Spring Boot 3.2.4

PostgreSQL

Spring Data JPA

Ez a szerepkör a rendszer gerincét építi fel. Az 1. fejlesztő felelős a projekt gyökerstruktúrájának kialakításáért, a Maven pom.xml függőségek (*Spring Web, WebSocket, Data JPA, PostgreSQL Driver, Lombok*) összeállításáért és az application.yml konfigurációs fájl megírásáért.

- Adatbázis Séma & Migráció:** PostgreSQL kapcsolat kiépítése, kapcsolatkezelő pool (HikariCP) beállítása, Flyway vagy JPA alapú automatikus séma-migrációs folyamatok.
- JPA Entitások Megvalósítása:** A ChargePoint (töltőoszlopok regisztrációja, vendor, modell, státusz mezők) és a ChargingSession (töltési munkamenetek, időbélyegek, fogyasztás, költség) entitások létrehozása relációkkal.
- Repository Interfészek:** Spring Data JPA repository-k megírása egyedi lekérdezésekkel a gyors adatbázis-hozzáférés érdekében.

2. Fejlesztő: OCPP Protokoll & Valós idejű Kommunikáció

WebSocket Server

OCPP 1.6-J

JSON Parsing

Hardware Simulation

A fizikai töltőoszlopok és a központi szerver közötti kétirányú, valós idejű kommunikációs réteg megvalósítása. A 2. fejlesztő biztosítja, hogy a platform szabványos OCPP 1.6-J (JSON over WebSockets) üzeneteket tudjon fogadni és küldeni.

- WebSocket Szerver Konfiguráció:** Aszinkron WebSocket endpointok beállítása Spring Boot alapon a töltők csatlakozásának kezelésére.
- OCPP Üzenetkezelő (*Message Parser*):** Bejövő JSON csomagok deszerializálása és routingja (BootNotification, StatusNotification, Heartbeat, StartTransaction.req, StopTransaction.req).
- Hardver Szimuláció Tesztelés:** Virtuális töltőoszlop kliensek indítása a kommunikációs csatornák stabilitásának és üzenetváltási helyességének igazolására.

Üzleti Logika és Integrációs Réteg

EV Charging Platform & Management System — Fejlesztési Útmutató (2. Rész)

3. Fejlesztő: Üzleti Logika, Hibakezelés & Self-Healing Alrendszer

Session Service State Machine Transaction Scheduler Self-Healing

A platform intelligens szíve, amely garantálja a rendszer magas rendelkezésre állását és kiküszöböli a piaci sztenderdekben gyakori kommunikációs hibákat és beragadt állapotokat.

- **Session Service & State Machine:** A töltési munkamenetek életciklusának kezelése (PENDING, ACTIVE, COMPLETED, FAILED) szigorú állapotátmeneti szabályokkal.
- **Árva Munkamenetek (*Orphan Sessions*) Detektálása:** Olyan hálózati vagy hardverhibák felismerése, ahol a kapcsolat megszakad, de a munkamenet hivatalosan nem zárult le.
- **Automatikus Öngyógyító (*Self-Healing*) Háttérfolyamatok:** Spring Boot @Scheduled ütemezők futtatása, amelyek periodikusan ellenőrzik a beragadt tranzakciókat, felszabadítják a töltőoszlopokat és kényszerített lezárást hajtanak végre.

4. Fejlesztő: Fizetési Gateway & REST API Integráció

REST Controllers Payment Gateway Pre-Authorization Void / Refund API

A külső felületek (mobilalkalmazás, webes front-end) és a pénzügyi tranzakciók biztonságos kezeléséért felelős modul. Megakadályozza a felhasználókat súlyosan sújtó beragadt banki zárolásokat.

- **REST API Végpontok:** Biztonságos végpontok biztosítása a felhasználói regisztrációhoz, töltésindításhoz, valós idejű adatok lekérdezéséhez és történethez.
- **Banki Pre-Authorization Integráció:** Fizetési kártya előzetes engedélyezése (*pre-auth*), de szigorúan csak akkor, amikor a fizikai hardver jelzi a kapcsolat stabilitását.
- **Azonnali Visszafizetési / Törlési Logika:** Automatikus Void / Refund API hívások kezelése timeout vagy sikertelen csatlakozás esetén, teljesen eliminálva a 10 000 Ft-os „pre-auth ghost hold” problémákat.

Összegzett Fejlesztési Szinergia

A négy modul szoros együttműködése biztosítja a vállalati szintű megbízhatóságot. Az 1. fejlesztő adatai alapozzák meg a 2. fejlesztő protokollkezelését, amelynek eseményei táplálják a 3. fejlesztő üzleti logikáját és állapotgépét, míg a 4. fejlesztő pénzügyi kapuja zárja be a biztonságos tranzakciós láncot.